MENTOR PRESENTATION GUIDE

MedFind

A full-stack prescription management platform connecting doctors, patients, and pharmacies in real time.

PROJECT NAME	STACK	PREPARED FOR	DATE
MedFind / Prescription Finder	React 19 · Firebase · Tailwind CSS	Mentor Review Session	March 2026

SECTION 01

What Is This Project?

MedFind is a web application that digitises the prescription process. Today, prescriptions are paper-based — doctors hand-write them, patients carry them to a pharmacy, and pharmacists manually look up stock. MedFind replaces every paper step with a real-time digital workflow.

The Problem It Solves

Lost Paper Prescriptions

Prescriptions get lost, damaged, or are illegible. MedFind stores them securely in the cloud.

Medicine Availability

Patients don't know which pharmacy stocks their medicine. MedFind shows real-time inventory.

Dose Tracking

Patients forget to take medication on time. The schedule feature shows a daily dose calendar.

Disconnected Systems

Doctors, patients, and pharmacies have no shared platform. MedFind connects all three.

The Core Idea — One Platform, Four Roles

Every user signs up and picks a role. The app then gives them a personalised dashboard with only the tools they need.

Doctor

- Write digital prescriptions
- Manage connected patients
- View patient history
- Browse connected pharmacies

Patient

- View own prescriptions
- Dose schedule / calendar
- Connect to a doctor
- Search medicine availability

Pharmacist

- Manage medicine inventory
- Fulfill prescription orders
- Track low-stock alerts
- Browse medicine database

Admin

- Manage all users
- Verify doctor accounts
- Access full system view
- Override any role page

SECTION 02

Technology Stack — What & Why

Every technology was chosen deliberately. When your mentor asks *"why did you use X?"* — use this table as your reference.

Technology	What it does	Why I chose it
React 19	Frontend UI library	Component-based, huge ecosystem, industry standard
Vite	Build tool & dev server	Extremely fast hot-reload, modern ESM approach
React Router v7	Client-side routing	Nested routes, lazy loading, role-based guards
Firebase Auth	User authentication	Email/password auth out-of-the-box, no backend server needed
Firebase Firestore	NoSQL cloud database	Real-time sync, scales automatically, free tier is generous
Tailwind CSS v4	Utility-first styling	Fast to style, consistent design tokens, no CSS file clutter
Radix UI / shadcn	Accessible UI components	Production-quality components (dialogs, dropdowns, etc.) with no extra design work
React Hook Form + Zod	Form handling & validation	Minimal re-renders, schema-based validation, great DX
Lucide React	Icon library	Clean, consistent SVG icons, tree-shakeable

Mentor Tip: If asked "why Firebase instead of a custom backend?" — answer: "As a frontend developer, Firebase let me focus on the product features (prescriptions, roles, inventory) instead of setting up REST APIs and a database server. It's a perfect fit for a portfolio project."

SECTION 03

Project Structure — How the Code Is Organised

The folder structure follows a **role-based architecture**. Code that belongs to a role lives inside that role's folder — this keeps the project easy to navigate and scale.

```
src/ ├── app/ │ ├── main.jsx ← Entry point (renders the app) │ ├── providers.jsx ← Wraps app with AuthProvider │ └── router.jsx ← All routes + role guards defined here │ ├── auth/ │ └── pages/ ← Login, SignUp, AuthCallback pages │ ├── context/ │ └── AuthContext.jsx ← Global auth state (user, profile, role) │ ├── guards/ │ └── RequireRole.jsx ← Blocks pages if wrong role │ ├── roles/ │ ├── doctor/ ← Everything the doctor sees │ │ ├── api.js │ │ ├── components/ │ │ └── pages/ │ ├── patient/ ← Everything the patient sees │ ├── pharmacy/ ← Everything the pharmacist sees │ └── admin/ ← Admin user management │ └── shared/ ├── components/ ← Reusable UI (layout, common, shadcn) ├── lib/ ← Firebase client, DB helpers, utilities └── pages/ ← Pages shared by multiple roles
```

Why This Structure?

Instead of grouping files by type (all components in one folder, all pages in another), files are grouped by **feature and role**. This means:

- ✓ Easy to find doctor-related code — it's all in `src/roles/doctor/`
- ✓ Adding a new role (e.g., "nurse") means adding one new folder
- ✓ Shared components are truly shared — no duplication
- ✓ The `guards/` folder ensures route-level security

SECTION 04

Key Features — Deep Dive

Feature 1 — Role-Based Authentication

When a user signs up, they choose their role (doctor, patient, pharmacy). This role is stored in Firestore under a `profiles` collection. Every time the user logs in, the app fetches this profile and stores it in **React Context** — making the role available everywhere in the app.

User logs in → Firebase Auth → Fetch profile from Firestore → Store in AuthContext → Route guard checks role → Correct dashboard shown

> The **RequireRole** guard wraps every protected route. If a patient tries to visit `/dashboard/new-prescription` (doctors only), they are redirected to the Unauthorized page automatically.

Feature 2 — Prescription Workflow

This is the core business flow of the app. Here is how a prescription travels from doctor to patient to pharmacy:

1. **Patient Connects to a Doctor**
   The patient sends a connection request. The doctor accepts it. Only connected patients appear in the doctor's patient list.

2. **Doctor Writes a Prescription**
   The doctor fills in: patient, diagnosis, one or more medicines (with dosage, frequency, duration), and optional notes. The form validates all fields before submission.

3. **Prescription Saved to Firestore**
   A prescription document is created along with individual prescription-item documents for each medicine. An expiry date is automatically calculated.

4. **Patient Views Their Prescriptions**
   The patient can open their prescriptions list and see all active and expired prescriptions with full details.

5. **Pharmacist Fulfills the Order**
   The pharmacist sees incoming prescription orders, checks their inventory, and marks the order as filled.

Feature 3 — Dose Schedule

Patients have a visual schedule page. Based on their active prescriptions, the app calculates which medicines they need to take each day and at what frequency. This turns raw prescription data into a personal medication calendar.

Feature 4 — Pharmacy Inventory System

Pharmacists manage their own medicine inventory. They can:

- ✓ Add medicines from the global medicine database to their inventory
- ✓ Set stock quantity and pricing
- ✓ See a dashboard with total items, low stock count, pending orders, and filled-today count
- ✓ Receive alerts when stock is running low

Feature 5 — Lazy Loading & Code Splitting

Not all pages are loaded when the app first opens. Using **React's lazy()** and **Suspense**, each role's pages are split into separate JavaScript bundles that only download when the user navigates to them. This makes the initial load fast.

```
// Router.jsx — pages load only when visited const NewPrescriptionPage = lazy(() =>
import("@/roles/doctor/pages/NewPrescriptionPage.jsx")) const SchedulePage = lazy(()
=> import("@/roles/patient/pages/SchedulePage.jsx")) // Wrapped in <Suspense> to
show a loading spinner while the page loads <Suspense fallback={<LoadingFallback
/>}> <NewPrescriptionPage /> </Suspense>
```

SECTION 05

How to Demo the App to Your Mentor

Before the meeting: Run `npm run dev` and have the browser open. Create test accounts in advance so you are not typing during the demo.

Recommended Demo Order (10–15 minutes)

1. **Start on the Login page (2 min)**
   Show the sign-in form. Explain that Firebase handles authentication securely. Log in as a **Doctor**.

2. **Doctor Dashboard (2 min)**
   Show the dashboard with quick-action cards and stats. Briefly mention the sidebar navigation and how menu items change based on role.

3. **New Prescription Page (3 min)**
   This is your **main feature**. Show selecting a patient, adding medicines with dosage & frequency, and submitting. Watch it redirect to the prescriptions list on success.

4. **Switch to Patient account (2 min)**
   Log in as a patient. Show the prescriptions list and dose schedule. Emphasise how the same prescription data is now viewed from the patient's perspective.

5. **Switch to Pharmacy account (2 min)**
   Show the inventory dashboard, medicines list, and orders. Point out the stats cards (total items, low stock, pending orders).

6. **Try accessing a blocked route (1 min)**
   While logged in as a patient, manually type `/dashboard/new-prescription` in the URL bar. Show the Unauthorized page — demonstrating the role guard works.

Tips for a Smooth Demo

- ✓ Use browser zoom (Ctrl + or Ctrl -) so your mentor can read the text
- ✓ Have test accounts already created before the meeting

- ✓ Keep browser DevTools closed unless asked — it looks cleaner
- ✓ If something breaks, stay calm — say "I know the issue, let me show you the code"
- ✓ Keep the project open in VS Code in one half of the screen if they want to see code

SECTION 06

Anticipated Mentor Questions & Answers

Q: Why did you choose Firebase instead of building your own backend API?

Firebase let me focus 100% on the frontend product features — prescriptions, user roles, inventory. Building a backend (Node.js + database + auth) would have doubled the scope of the project. Firebase is also production-grade, used by thousands of companies, so it's not "cheating" — it's a pragmatic choice.

Q: How does your role-based access control work?

Each user has a "role" field in their Firestore profile (doctor, patient, pharmacy, admin). After login, I fetch this profile and store it in React Context. Every protected route is wrapped in a RequireRole component that reads the role from context. If the role doesn't match, the user is redirected to an Unauthorized page.

Q: What happens if a user tries to bypass the route guard by changing the URL?

The RequireRole guard runs on every render, not just on navigation. Even if a user manually types a URL, the guard checks their actual role from context before rendering the page. The Firestore security rules also enforce this on the data level.

Q: How do you handle loading states?

Auth state loading is tracked in AuthContext. Lazy-loaded pages show a LoadingFallback spinner via React Suspense. Individual data fetches inside components track their own loading state with useState.

Q: What would you improve if you had more time?

I would add: (1) real-time notifications when a prescription is created, (2) a chat feature between doctor and patient, (3) PDF export for prescriptions, (4) proper unit and integration tests, (5) a mobile app using React Native.

Q: How is the project structured and why?

I use a role-based folder structure where each role (doctor, patient, pharmacy, admin) has its own folder with its own components, pages, and API calls. Shared components live in a shared folder. This makes it easy to add a new role without touching existing code.

Q: Did you write this alone? How long did it take?

Yes, this is my own project. I used Claude Code (an AI assistant) to help with boilerplate and debugging, similar to how developers use Stack Overflow or GitHub Copilot. All architectural decisions and feature design are my own.

SECTION 07

Presentation Script — What to Say

This is a suggested script. Speak naturally — do not read it word for word. Use it to prepare and practice.

Opening (30 seconds)

"The project I built is called MedFind — a prescription management platform. The core idea is to digitise the whole prescription workflow: a doctor writes a prescription digitally, the patient can immediately see it on their phone, and pharmacies can check their inventory against incoming prescriptions. I built the entire frontend with React, and I used Firebase for the backend — authentication and the database."

Explaining the Architecture (1–2 minutes)

"The app has four user roles — doctor, patient, pharmacist, and admin. Each role gets a different dashboard with different features. I handle this with a role-based access system. When a user logs in, their role is fetched from Firestore and stored in React Context. Every route is protected by a 'RequireRole' guard that checks whether the current user has permission to see that page."

Showing the Prescription Feature (during demo)

"Here I'm logged in as a doctor. I'll go to New Prescription. I select the patient — notice only connected patients appear here, so I can't accidentally prescribe for someone I don't know. I add the medicines, set dosage and frequency, and submit. The prescription is saved to Firestore with an auto-calculated expiry date. If I switch to the patient's account, they can already see this prescription in their dashboard."

Closing (30 seconds)

"The biggest thing I learned from this project is how to architect a multi-role application — making sure different users have the right access and the right experience without duplicating a lot of code. I'd love your feedback on the structure and any areas you think I should improve."

Golden Rule: Always end by asking your mentor a question. It shows you are engaged and open to

> feedback — and mentors love that. Try: "*What's the first thing you would refactor in this project?*"

SECTION 08

Pre-Meeting Checklist

The Day Before

- [ ] Run `npm run dev` and make sure the app starts without errors
- [ ] Create at least 2 test accounts: one doctor, one patient, one pharmacy
- [ ] Create a sample prescription so there is real data to show
- [ ] Add some medicines to the pharmacy inventory
- [ ] Make sure your internet connection is stable (Firebase needs internet)
- [ ] Read sections 4 and 6 of this guide at least twice
- [ ] Practice the demo once alone, timing yourself

On the Day

- [ ] Start `npm run dev` before the meeting begins
- [ ] Keep your browser zoom at a comfortable level (120% works well)
- [ ] Close all other browser tabs to avoid distractions
- [ ] Have VS Code open with the project in case your mentor wants to see code
- [ ] Keep this PDF guide open on a second monitor or phone for quick reference
- [ ] Bring a notepad to write down feedback

Key Files to Know by Heart

File	What it does
src/app/router.jsx	All routes and role guards defined here
src/context/AuthContext.jsx	Global auth state — user, profile, role

File	What it does
src/guards/RequireRole.jsx	Component that blocks unauthorised access
src/roles/doctor/components/new-prescription-form.jsx	The core prescription-writing feature
src/roles/pharmacy/components/pharmacist-dashboard.jsx	Pharmacy stats and quick actions

> **Final Reminder:** Your mentor is there to help you grow — not to judge you. If you don't know the answer to something, say "*That's a great question. I haven't explored that yet, but I'll look into it after this meeting.*" Honesty is always better than guessing.

MedFind — Prescription Finder | Mentor Presentation Guide | Prepared March 2026